

#3) Data Types & Type Casting

Common Data Types (dtypes):

int8, int32, int64 — Integers
float32, float64 — Floating point (Very Important)
bool, complex

Type Casting:

```
arr = np.array([1, 2, 3])
arr_float = arr.astype(float)
arr32 = np.float32(arr) # Memory efficient for Deep Learning
```

Key Point: Use **float32** instead of **float64** in Deep Learning to save memory.

#4) Indexing & Slicing

Basic Indexing:

```
arr[0] # first element
arr[-1] # last element
arr[1:5] # slicing
arr[::2] # every second element
```

2D Indexing:

```
arr[0, 1] # row 0, column 1
arr[0:2, 1:3] # submatrix
arr[:, 0] # first column
```

Fancy Indexing:

```
arr[[0, 2, 4]]
```

Multi-dimensional slicing is heavily used in Images and Tensors.

#5) Array Operations & Math

Element-wise Operations:

```
+, -, *, /, **, //, %
```

Math Functions:

```
np.sqrt(), np.exp(), np.log(), np.log10()
np.abs(), np.ceil(), np.floor(), np.round()
```

Matrix Operations:

```
np.dot(a, b) or a @ b
np.matmul(a, b)
```

np.dot() and **@** are the heart of Neural Network forward passes.

#6) Aggregation & Statistics

Basic Stats:

```
np.sum(), np.mean(), np.median(), np.min(), np.max(), np.std(), np.var()
```

Axis Parameter (Very Important):

```
np.sum(arr, axis=0) # column-wise
np.sum(arr, axis=1) # row-wise
```

axis= parameter is asked in almost every ML interview.

#7) Shape Manipulation

Reshape & Transpose:

```
arr.reshape(3, 4)
arr.T # Transpose
arr.flatten()
```

Combine Arrays:

```
np.concatenate(), np.stack()
np.vstack(), np.hstack()
```

reshape() and **expand_dims()** are used daily in ML for batch dimensions.

#8) Boolean Indexing & Filtering

Filtering:

```
arr[arr > 5]
arr[(arr > 2) & (arr < 8)]
```

Useful Functions:

```
np.where(), np.any(), np.all()
```

NaN Handling:

```
np.isnan(), np.nan_to_num()
```

np.nan_to_num() is critical for cleaning real-world ML datasets.

#9) Broadcasting

Concept: NumPy automatically expands smaller arrays to match larger ones during operations.

Understanding Broadcasting = Understanding how TensorFlow & PyTorch work internally.

Practical Uses: Adding bias, normalization, scalar operations.

#10) Sorting & Searching

Sorting:

```
np.sort(), np.argsort()
```

Searching:

```
np.argmax(), np.argmin(), np.nonzero()
```

np.argmax() is used in every classifier to get predicted class.

#11) Linear Algebra (np.linalg)

Important Functions:

```
np.linalg.norm(), np.linalg.inv(), np.linalg.det()
np.linalg.svd(), np.linalg.eig()
np.linalg.solve()
```

SVD and eig() are important to understand PCA.

#12) Copying & Memory

```
arr.copy() — Deep Copy (Safe)
arr.view() — Shallow View
```

Always use **copy()** to avoid silent bugs in ML pipelines.

#13) Vectorization & Performance

Avoid Python loops. Use vectorized operations and broadcasting.

Vectorized code is 10–100x faster.

#14) ML-Specific Utility Functions

```
np.clip() — Gradient Clipping
np.sign(), np.bincount(), np.histogram()
```

#15) Set Operations (Nice to Know)

```
np.union1d(), np.intersect1d(), np.setdiff1d(), np.isin()
```

#16) Saving & Loading Data

```
np.save('file.npy', arr)
arr = np.load('file.npy')
```

.npy format is fastest for ML data.

Golden Rule 🌟

Don't just read — Open Jupyter Notebook and type every function yourself!

Break it → Fix it → Master it